



Computer Vision

BSc AIDAMS ESSEC/CentraleSupélec
IDSI51006 - Winter 2026
Titouan Rigaud

How to contact me ?

Titouan Rigaud

Data engineer at CASD

email : titouan.rigaud@casd.eu

Schedule

1. **Today (7th sep)** : 9am - 12am
2. **Today (7th sep)**: 1pm - 4pm
3. **Tomorrow (8th sep)** : 9am - 12am
4. **Tomorrow (8th sep)** : 9am - 12am
5. **29 sep**: 9am - 12am
6. **29 sep** : 1pm - 4pm
7. **30 sep** : 9am - 12am

Schedule

1. **Today (7th sep)** : 9am - 12am
2. **Today (7th sep)**: 1pm - 4pm
3. **Tomorrow (8th sep)** : 9am - 12am → **10 minute quiz**
4. **Tomorrow (8th sep)** : 1pm - 4pm
5. **29 sep**: 9am - 12am
6. **29 sep** : 1pm - 4pm
7. **30 sep** : 9am - 12am → **10 minute quiz**

No final exam, instead you will complete a project and a report.

Grading policy

Quizzes	**Quiz 1**	**Quiz 2**	**Total**
	20%	20%	40%
Kaggle Challenge	**Code**	**Report**	**Total**
	20%	40%	60%
		Total	100%

More on them later !

A few rules for this class…

- There are many questions asked during the class :
  - Wrong answers are welcome ! They help everyone learn.
  - If no one answers, I might choose someone like last year.
  - If you experience strong social anxiety, feel free to email me.
- Respect is essential :
  - Everyone deserves to feel comfortable speaking up.
  - Make sure you listen to your classmates.
  - Mocking or disrespectful behaviour will be reported.
- I trust you to manage yourselves :
  - I do take attendance, but you can leave afterwards.
  - You may exit the room at any time without asking for my permission.
  - I may also ask you to exit the room if you are disturbing the class.
  - You are responsible for catching up on what you miss - but you can ask me questions

Outline for today

01. Course details ✅
02. Introduction
03. Standard Image Processing
    (and why we will never use it again !)
04. Machine Learning for Images
    (and why we will never use it again !)
05. Registration and first look at the Kaggle platform
06. Deep Learning for Images

Outline for today

01. Course details ✅
02. **Introduction**
03. Standard Image Processing
    (and why we will never use it again !)
04. Machine Learning for Images
    (and why we will never use it again !)
05. Registration and first look at the Kaggle platform
06. Deep Learning for Images

Who here likes pasta ?

What is pasta ?



Pastas may have different shapes

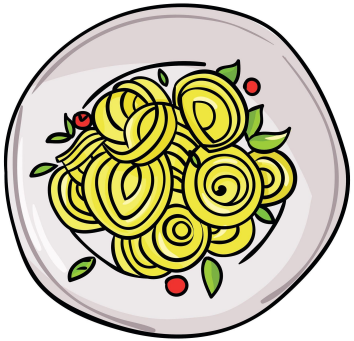


They also may have different colours



They may not contain the same ingredients !

Is this still pasta ?



Yes ?



Hmm… maybe ?



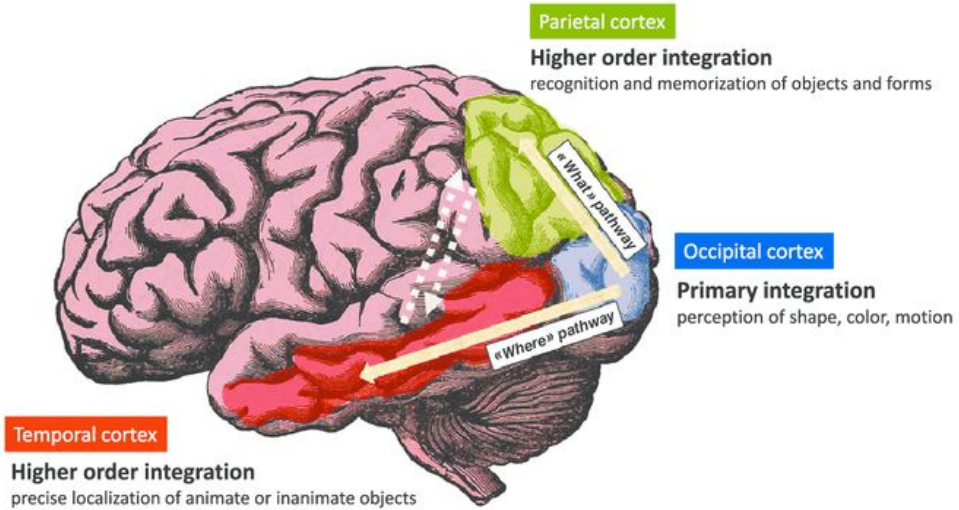
Is this a cake ??

How do we know what is pasta ?

When we see pasta…

- We do not just “see” shapes and colours.
- Our brain interprets this image based on :
  - 👁️ The previous times we **saw** pasta
  - 👃 The previous times we **smelled** pasta
  - 👂 The previous times we **heared** people talk about pasta
  - 👄 The previous times we **tasted** pasta
  - 👌 The previous times we “**touched**” pasta ?
- We learnt over time the **concept of pasta**. 🍝
- This concept may have shifted as we grew up (*are gnocchis even considered pasta* ?)

When we see pasta…



Source : Clonal selection versus clonal cooperation: the integrated perception of immune objects, Serge Nataf (2016)

Description : Brain visual pathways. The signals related to the shape, color and motion of visual objects are integrated by specialized brain neuronal populations residing in the primary visual cortex i.e the superficial neuronal layers of the brain occipital lobe. Output signals are then generated that instruct other cortical areas for higher order integration tasks. The " What " pathway connects the primary visual cortex to areas of the temporal cortex that are essential to the recognition and memorization of objects and forms. The " Where " pathway connects the primary visual cortex to areas of the parietal cortex that support perception of precise localization. The interconnections between higher-order visual areas (dashed arrows) as well as other brain areas not highlighted here, allows a fully-integrated perception that takes into account the " What", " Where ", " How " and " When " features of a visual object.

How do computers “see” pasta ?

For computer, images are just matrices…



Handwritten digit (8) from MNIST dataset

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0													
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29													
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0													
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1													
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49													
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36													
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62													
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0													
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0													
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19													
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0													
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0													
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4													
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0													
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0													
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3													
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0													
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4													
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5													
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0													
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1													
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0													

Matrix representation with color values

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0													
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29													
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0													
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1													
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49													
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36													
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62													
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0													
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0													
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19													
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0													
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0													
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4													
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0													
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0													
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3													
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0													
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4													
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5													
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0													
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1													
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0													

Matrix representation

And these matrices are just numbers…

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 255 & 0 & 0 \\ 255 & 0 & 255 & 0 \\ 180 & 0 & 180 & 0 \end{pmatrix}$$

- When using *numpy* in Python, there is a **raw memory** that stores the numbers in a line, and then the metadata helps shape the matrix by giving the **data type** (is it an integer ? or a floating number ?) and its **shape** (how many rows ? columns ?)
- Here we would have in the memory :
  0 0 0 0 0 0 255 0 0 255 0 255 0 180 0 180 0
- And the metadata would contain information like :
  shape = (4, 4), type = uint8

uint8 ???

These numbers are just… 0 and 1 ?

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 255 & 0 & 0 \\ 255 & 0 & 255 & 0 \\ 180 & 0 & 180 & 0 \end{pmatrix}$$

- For most computers, each memory cell is set up to store lists of 0 and 1.
- They are grouped in eight bits (called a byte).
- Each byte is able to represent 256 different numbers ($2^8 = 256$).
- So the above matrix looks like this for a computer :
  00000000 00000000 00000000 00000000 00000000 00000000 11111111 00000000 00000000 11111111 00000000 11111111 00000000 10110100 00000000 10110100 00000000
  (without spaces!!!)

And if we want to store higher numbers ?
Or negative numbers ?

This is not today’s goal !

Our pixels will always range from 0 to 255.

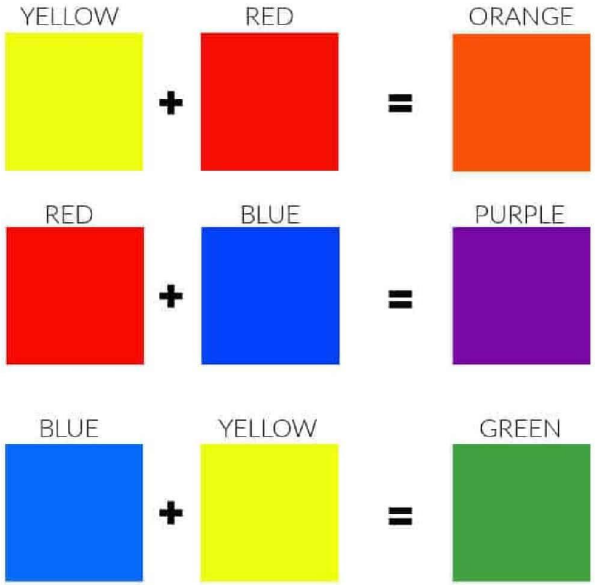
But if you are curious, you already learnt this last year in “Fondamentals of Cloud Computing” 😉

And what about coloured images ?

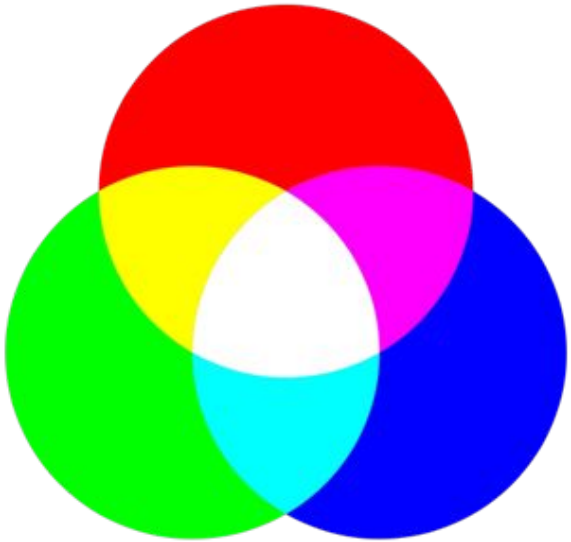
Outline for today

01. Course details ✅
02. Introduction ✅
03. **Standard Image Processing**
    (and why we will never use it again !)
04. Machine Learning for Images
    (and why we will never use it again !)
05. Registration and first look at the Kaggle platform
06. Deep Learning for Images

A bit of colour theory



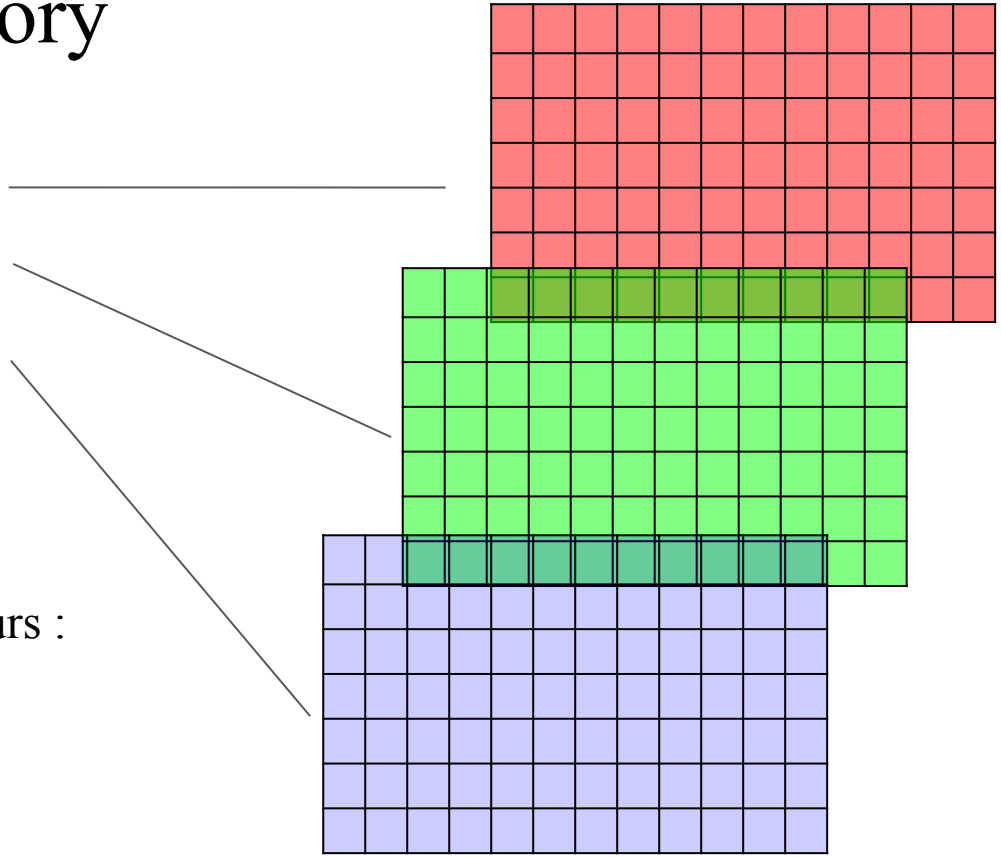
Remember when you painted as a kid ?



Well, with computer screens, things are different…

A bit of colour theory



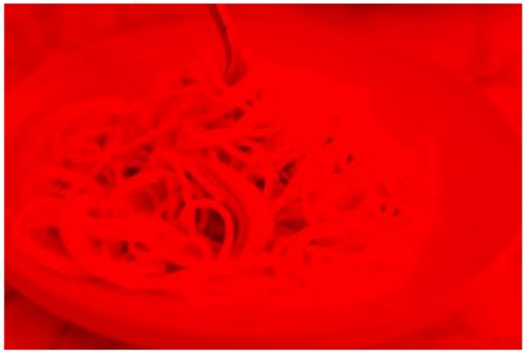


Each image is represented by 3 matrices for the 3 primary colours :

- Red
- Green
- Blue

A bit of colour theory

Just the red channel



Just the green channel



Just the blue channel



Pasta has a lot of yellow, so there is a lot of green and red.

However, there is little blue (so it appears darker)

Let’s start coding !

1. Download “Demo 1 - Standard Image Processing” on Moodle
2. Unzip the folder
3. Open in VSCode, Pycharm or Colab

Standard image processing

- Most digital image processing techniques were developed in the 1960s.
- Based on matrix operations and signal processing theory, their main goals were to :
  - Improve the quality of an image (denoising, restoration)
  - Extract features (edges, corners, colours)
  - Segment images
  - Classify images
  - Recognise patterns
- Main domains of applications : medical imaging, astronomy, satellite photography

How can we denoise an image ?

How to denoise images ?

Blurred images have less sharp edges and are less detailed, but the bigger shapes remain visible. This allows to remove noise from the image.

Image with noise



Denoised image



How to blur images ?

There are many techniques, but we will see the most well known one :

Gaussian blur

To achieve it, we need to :

- average each pixel value with the values of its neighbouring pixels
- give more weights to pixels which are closer to the centre

In image processing, we can apply this kind of transformation by using **convolution kernels**. This is called “filtering”.

What is a convolution kernel ?

A convolution kernel is a matrix with a size smaller than the original image. It gives the weights to apply for a weighted sum. For Gaussian blur, a convolution kernel might look like :

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

What does it look like ?

1. Multiply the 1st neighbor pixel by the 1st value in the Kernel

125	213	98	203	202	170
104	145	161	204	201	157
72	8	209	202	194	144
73	9	202	201	194	156
81	15	189	185	181	144
15	189	185	194	227	158

Original Image

1	2	1
2	4	2
1	2	1

3x3
Gaussian Kernel

Source : https://medium.com/hackernoon/cv-for-busy-developers-convolutions-5c984f216e8c

Let’s try it on the notebook !

How can extract features from an image ?

Detecting edges using Sobel algorithm

Goal : We want to detect edges by measuring intensity changes in an image.
Idea : Compute gradients in both *x* and *y* directions.
Steps (simplified) :

1. Convert the image to grayscale.
2. Convolve the image with two 3×3 Sobel kernels (see opposite matrices). You obtain G_x and G_y
3. Compute the gradient magnitude for each pixel :
   $G = \sqrt{G_x^2 + G_y^2}$
4. Get the directions of the gradients (using the arctangent of the ratio)
5. Highlight strong gradients → these correspond to **edges** !

The **Canny** adds extra steps to smooth and refine edges detection, but you are not expected to know the details.

x-direction kernel

-1	0	1
-2	0	2
-1	0	1

y-direction kernel

-1	-2	-1
0	0	0
1	2	1

Why does Sobel work ?

Sobel amplifies light changes in any direction : it allows to increase the variation and identify where these variations are the strongest. These changes are the edges.



Let’s try it on the notebook !

And now what if we want to detect corners ?

Detecting corners using Harris algorithm

Goal : detect corner points, *i.e.* pixels where intensity changes in all directions.
Idea : while edges have a strong gradient in one direction, corners have strong gradients in both x and y directions.
Steps (simplified) :

1. Compute the image gradients Gx and Gy (for example using Sobel filters).
2. For each pixel, use a small window to summarise local gradient variations in a matrix M.
3. Compute the Harris response:
   $R = \det(M) - k \cdot (\text{trace}(M))^2$ (where k is a small constant, e.g. 0.04–0.06)

det(M) is the product of the eigenvalues → it becomes large when both directions vary.
trace(M) is their sum → it helps correct cases where only one direction varies (edge).

So if R is:

- large and positive → corner
- negative → edge
- close to zero → flat region

Let’s try it on the notebook !

And what about segmentation ?

Segmenting images 101

We can simply… apply a threshold on a grayscale image !

Of course, more advanced techniques exist, if you are curious you can check out this notebook : https://www.kaggle.com/code/mielek/image-segmentation-with-opencv

Let’s try it on the notebook !

Outline for today

01. Course details ✅
02. Introduction ✅
03. Standard Image Processing ✅
    (and why we will never use it again !)
04. **Machine Learning for Images**
    (and why we will never use it again !)
05. Registration and first look at the Kaggle platform
06. Deep Learning for Images

Machine learning VS Image processing

⚠️ The following slides are only supposed to be a reminder from previous courses you attended to, it is not self-sufficient !



Machine learning VS Image processing

⚠️ The following slides are only supposed to be a reminder from previous courses you attended to, it is not self-sufficient !

Image processing



Image,
Rule

Rule :
Convolution
Algorithm…

Image

Machine learning VS Image processing

⚠️ The following slides are only supposed to be a reminder from previous courses you attended to, it is not self-sufficient !

(supervised) Machine Learning



Image, Labels

Learned model : Logistic Regression, trained KNN…

Predicted labels

Machine learning VS Image processing

⚠️ The following slides are only supposed to be a reminder from previous courses you attended to, it is not self-sufficient !

(unsupervised) Machine Learning



Image

Learned model :
K-Means
DBSCAN…

Cluster

Machine learning VS Machine learning for images

When doing machine learning on tabular data, we carefully select our inputs :

name	age	gender	annual_income	nb_loans	requested_amount	grant_loan ?
alice	48	F	60 000	0	300 000	yes
bob	36	M	40 000	3	350 000	no
…						
zineb	25	F	33 000	0	10 000	yes

But what about images ? Can we just give the raw pixels as an input ?

Let’s start coding !

1. Download “Demo 2 - Machine learning for images” on Moodle
2. Unzip the folder
3. Open in VSCode, Pycharm or Colab

What if we used image processing to manually create features ? Which ones could be useful ?

Outline for today

01. Course details ✅
02. Introduction ✅
03. Standard Image Processing ✅
    (and why we will never use it again !)
04. Machine Learning for Images ✅
    (and why we will never use it again !)
05. **Registration and first look at the Kaggle platform**
06. Deep Learning for Images

Time for a recap…



https://www.menti.com/alpn5k9k3ibs

Introduction to Kaggle

Links :

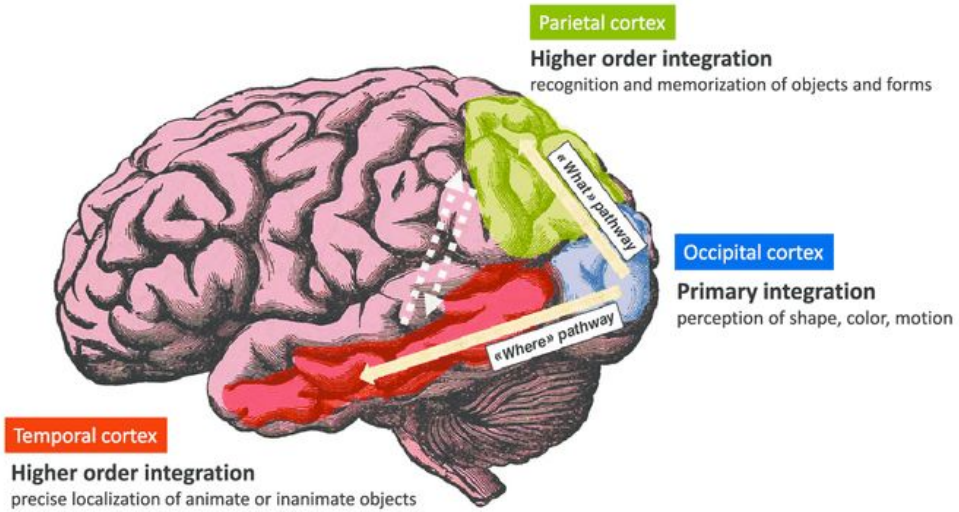
- To join the platform : https://www.kaggle.com/t/ce6c628183e3487ebccbc1aa9fa53caa

Outline for today

01. Course details ✅
02. Introduction ✅
03. Standard Image Processing ✅
    (and why we will never use it again !)
04. Machine Learning for Images ✅
    (and why we will never use it again !)
05. Registration and first look at the Kaggle platform ✅
06. **Deep Learning for Images**

A brief history of deep learning

Remember this image ?

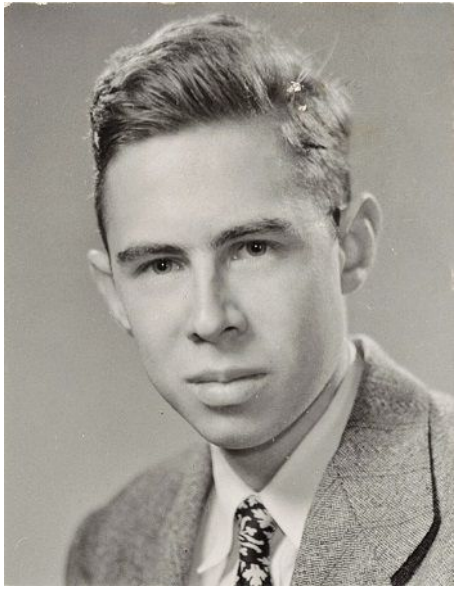


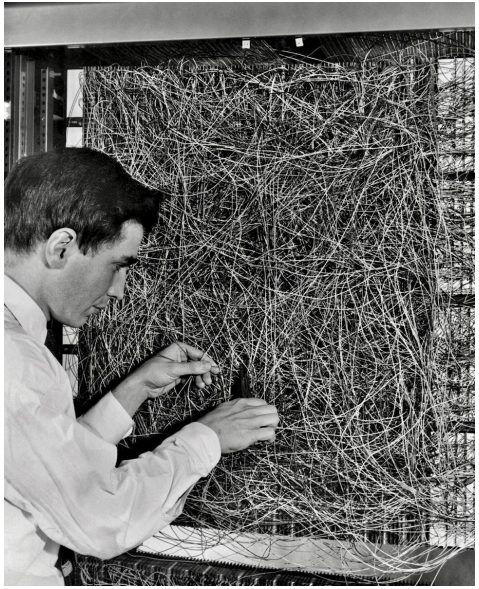
A brief history of deep learning

The first neural network was actually just a neuron, called the **perceptron**.

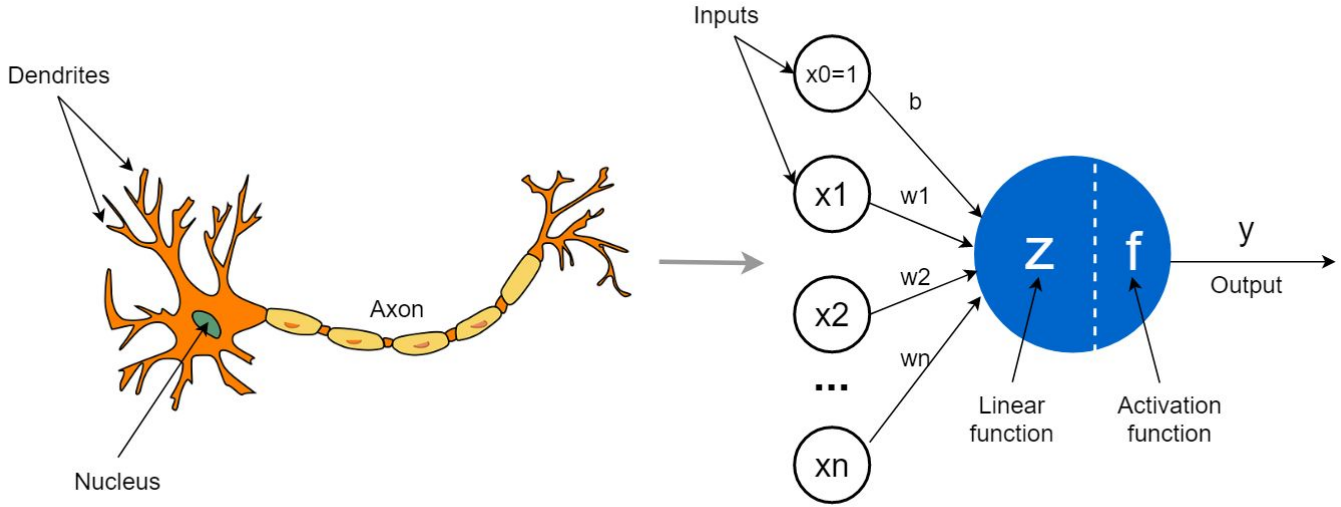
It was invented in 1957 by **Frank Rosenblatt**.

He designed the Mark I Perceptron, a computer able to learn a model to recognise visual shapes by adjusting the weights of a perceptron.





The perceptron



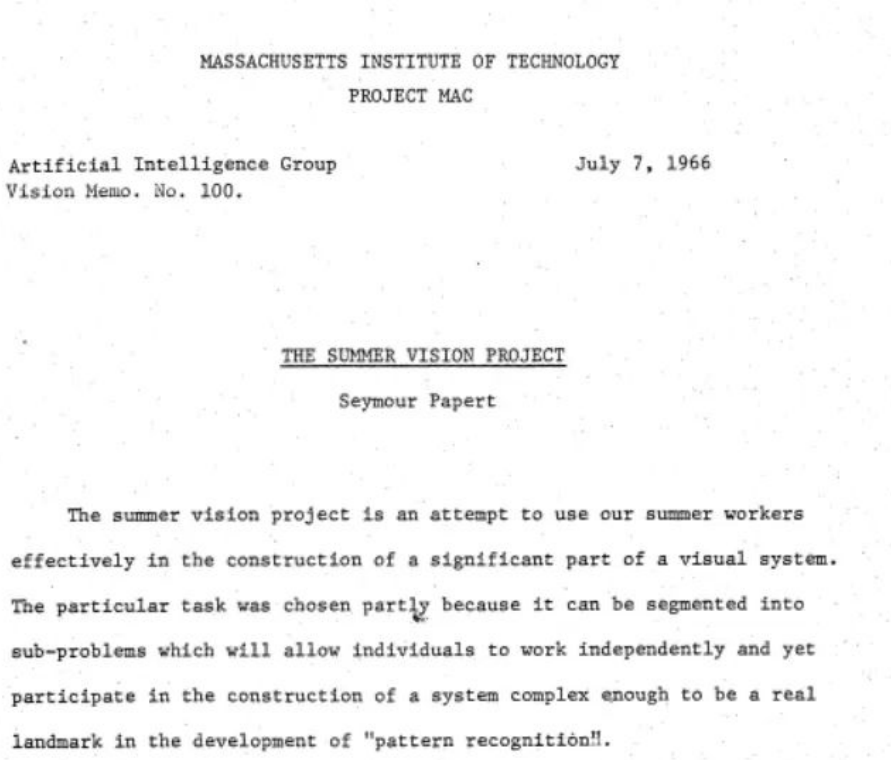
Source : https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc/

A brief history of deep learning

Originally, computer vision didn't seem that "hard".

In 1966, **Seymour Papert**, an MIT professor, aimed to address the challenge of computer vision within a short span of a few months, during what he called the **Summer Vision Project**.

60 years later, computer vision is still a challenge.



MASSACHUSETTS INSTITUTE OF TECHNOLOGY
PROJECT MAC

Artificial Intelligence Group — July 7, 1966
Vision Memo. No. 100.

THE SUMMER VISION PROJECT

Seymour Papert

The summer vision project is an attempt to use our summer workers effectively in the construction of a significant part of a visual system. The particular task was chosen partly because it can be segmented into sub-problems which will allow individuals to work independently and yet participate in the construction of a system complex enough to be a real landmark in the development of "pattern recognition".

Challenges at the time

In 1969, Seymour Papert and Marvin Minsky published **Perceptrons**. Their goal was to show Frank Rosenblatt's perceptron strong limitations.

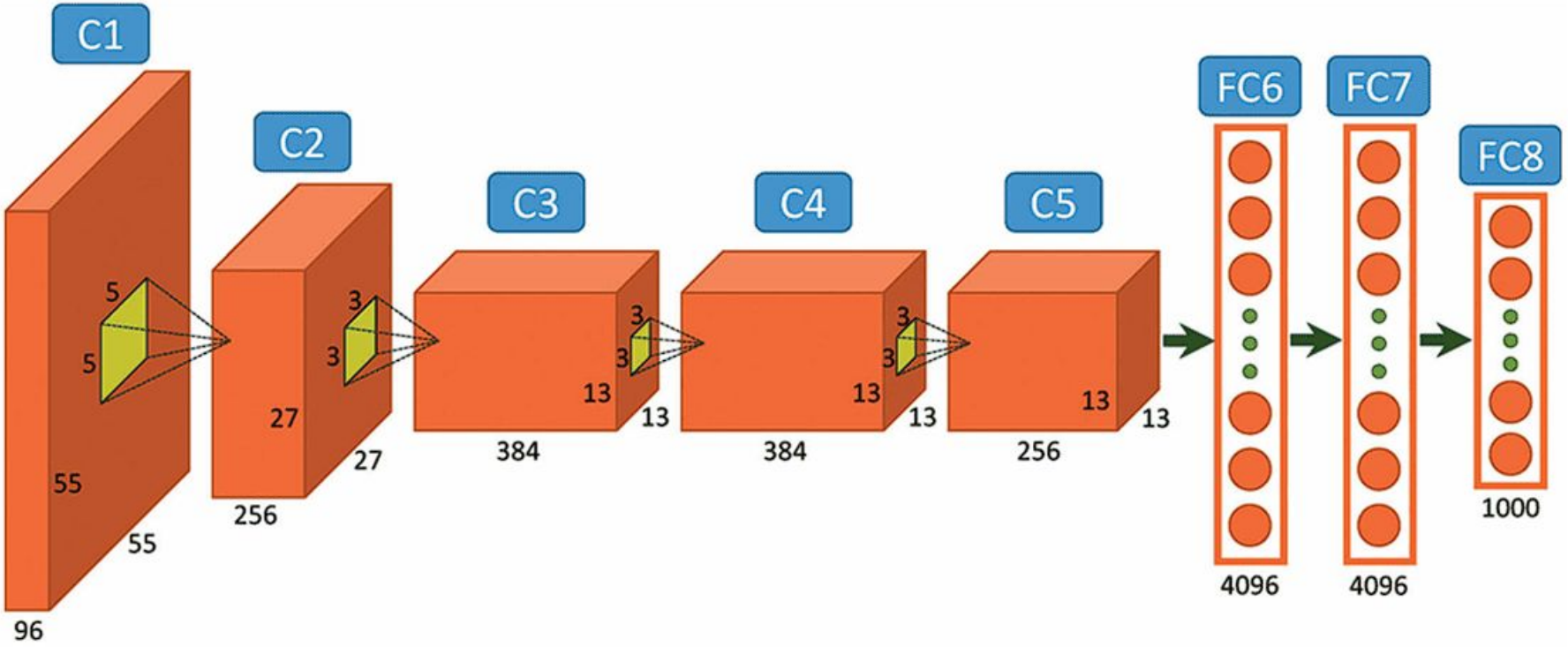
At the time, perceptrons were not that successful because there was :

- not enough data
- not enough computation power

The pessimism of the authors lead to a strong shift in the AI research directions, which lead to the **AI winter** in the 1980s.

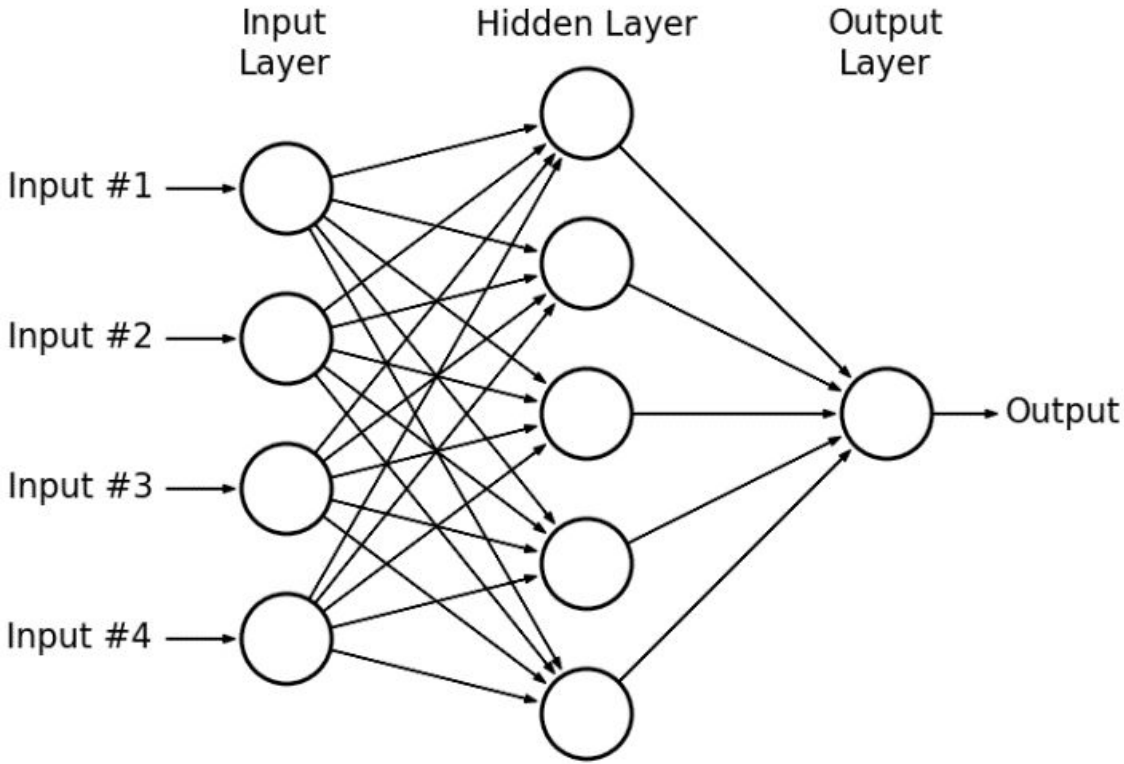
The years passed, until…

AlexNet (2012), or the beginning of Deep Learning Revolution



Wait wait wait… we missed a few steps, right ?

The Multi-Layer Perceptron



Let’s play !

https://playground.tensorflow.org/

Outline for today

01. Course details ✅
02. Introduction ✅
03. Standard Image Processing ✅
    (and why we will never use it again !)
04. Machine Learning for Images ✅
    (and why we will never use it again !)
05. Registration and first look at the Kaggle platform ✅
06. Deep Learning for Images ✅

For the quiz next time…

- You will get 10 minutes to answer 2 open questions regarding today's course.
- Topics of the question :
  - Image processing
  - Machine learning for images
  - ~~Deep learning for images~~
- If you cannot attend, please send me a justification by email or you will get 0.
- Reminder : this quiz will count for 20% of you grade.

See you in next week !